

Rules, Custom Entities & Chatbots

Pre-Assessment 4 · 8 questions · 31 points

Name: _____

Date: _____

Score: / 31

This is a **pre-assessment**. It shows what you already know **before** we start this unit, so it is fine if some questions cover things you have not learned yet. Try every question you can. If you have no idea at all, write **“not yet”** and move on.

Work on your own, without a computer. You have about **40 minutes**. When a question asks what a program prints, write exactly what Python would print, one line of output per answer line. If the code would crash, write the name of the error (for example, `TypeError`). Show your thinking; partial credit is given.

In every spaCy question, assume this code has already run: `import spacy` and `nlp = spacy.load("en_core_web_sm")`

1. Does it match?

4 points

```
pattern = [{"LOWER": "hello"}, {"IS_PUNCT": True}]
```

For each text, write **Yes** if the pattern finds a match, or **No** if it doesn't.

Text	Match?
"Hello!"	
"hello there"	
"HELLO, friend"	
"Say hello"	

2. Write the pattern

3 points

Write a Matcher pattern (a list of dictionaries) for each job.

(a) The word "pizza" with any capitalization.

(1 pt)

(b) The phrase "tell me a joke" with any capitalization.

(1 pt)

(c) The word "my" followed by **any noun** (like "my dog").

(1 pt)

3. Rule-based matching

4 points

```
from spacy.matcher import Matcher
matcher = Matcher(nlp.vocab)
matcher.add("GREETING", [[{"LOWER": "hi"}], [{"LOWER": "hello"}]])
doc = nlp("Hi there! Hello, hello?")
matches = matcher(doc)
for match_id, start, end in matches:
    print(start, end, doc[start:end].text)
```

The tokens are: Hi(0) there(1) !(2) Hello(3) ,(4) hello(5) ?(6)

(a) How many matches are found? (1 pt)

(b) Write the line printed for the **second** match. (2 pts)

(c) What is `match_id`, and how do you turn it back into the name "GREETING"? (1 pt)

4. Privacy Shield

4 points

`t.ent_iob_` is **B** for the first token of an entity, **I** for a token inside one, and **O** for a token outside every entity. `t.ent_type_` is the token's entity label ("" if none).

```
doc = nlp("Ava Stone lives in Chicago.")
out = ""
for t in doc:
    if t.ent_type_ == "PERSON":
        if t.ent_iob_ == "B":
            out = out + "[PERSON] "
        else:
            out = out + t.text_with_ws
print(out)
```

(a) spaCy labels "Ava Stone" PERSON and "Chicago" GPE. Fill in `ent_iob_` for each token. (2 pts)

Token	Ava	Stone	lives	Chicago
<code>ent_iob_</code>				

(b) What does the program print? (2 pts)

5. Teaching spaCy new entities

4 points

```
ruler = nlp.add_pipe("entity_ruler", before="ner")
patterns = [
    {"label": "FOOD", "pattern": "taco"},
    {"label": "FOOD", "pattern": [{"LOWER": "ice"}, {"LOWER": "cream"}]},
]
ruler.add_patterns(patterns)
doc = nlp("Mia ate a taco and Ice Cream in Denver.")
print([(e.text, e.label_) for e in doc.ents])
```

(a) What does it print? (The model also finds a PERSON and a GPE.) (2 pts)

(b) Would the first pattern match the word "Taco" with a capital T? Why? (1 pt)

(c) What does `before="ner"` do? (1 pt)

6. Fantasy Map Finder

3 points

On its own, spaCy's model labels **Mordor** in "Frodo walked to Mordor." as **ORG**. A student adds the rule `{"label": "PLACE", "pattern": "Mordor"}` with an entity ruler.

(a) If the ruler is added with `before="ner"`, what label does Mordor get? (1 pt)

(b) If the ruler is added with `after="ner"`, what label does Mordor get? (1 pt)

(c) Why are rules especially useful for fantasy stories? (1 pt)

7. KidBot Brain

4 points

```
from spacy.matcher import Matcher
matcher = Matcher(nlp.vocab)
matcher.add("GREETING", [[{"LOWER": "hi"}], [{"LOWER": "hello"}]])
matcher.add("WEATHER", [[{"LOWER": "weather"}]])

def reply(text):
    doc = nlp(text)
    matches = matcher(doc)
    if matches:
        intent = nlp.vocab.strings[matches[0][0]]
        if intent == "GREETING":
            return "Hi! I'm KidBot."
        if intent == "WEATHER":
            places = [e.text for e in doc.ents if e.label_ == "GPE"]
            if places:
                return f"It's sunny in {places[0]}!"
            return "Which city?"
    return "Sorry, I don't understand yet."
```

Write what `reply(...)` returns for each message. (spaCy finds "Paris" as a GPE.)

Message	KidBot replies
"Hello!"	
"What's the weather in Paris?"	
"What's the weather like?"	
"Tell me a joke"	

8. Joke intent

5 points

Write code that creates a `Matcher` with a rule named `"JOKE"` that matches either the word `"joke"` or the phrase `"make me laugh"` (any capitalization). Then write a function `wants_joke(text)` that **returns** `True` if the text matches and `False` if it doesn't.

END OF ASSESSMENT